UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



TECNOLOGÍA DE OBJETOS

ACTIVIDAD N° 12

Estudiantes:

- Schreiber Landeo Diego Hans
- Ramos Ochochoque Elkin Eder B.
- Solsol Choque Fernando Rafael
- Huaynacho Mango Jerry Anderson
- Quispe Mamani Jose Gabriel

DOCENTE: MARIBEL MOLINA BARRIGA

AREQUIPA – PERÚ
2026

1. Introducción

Los patrones de diseño ofrecen soluciones conocidas para problemas frecuentes al organizar software. Singleton es un patrón creacional que permite que una clase tenga una sola instancia y proporciona un punto de acceso común a ella. En esta actividad se aplica a una clase de configuración que concentra preferencias de una aplicación de escritorio.

2. Objetivos

- Aplicar el patrón Singleton en C++.
- Centralizar preferencias globales de tema, volumen y notificaciones.
- Usar clases, encapsulamiento y control de creación de objetos.
- Explicar brevemente el funcionamiento del patrón mediante un ejemplo ejecutable.

3. Descripción del problema

En una aplicación, distintas partes pueden necesitar consultar o modificar las mismas preferencias. Si cada parte creara su propio objeto de configuración, los valores podrían ser diferentes y producir resultados inconsistentes. Singleton evita esta situación al hacer que todas las partes accedan a un único objeto compartido.

4. Diseño de la solución

La clase Configuracion contiene tres preferencias: tema, volumen y estado de las notificaciones. Su constructor privado impide crear objetos desde fuera de la clase. El método estático obtenerInstancia() devuelve una referencia a una variable estática local, creada una sola vez. Los métodos de acceso permiten mantener los atributos privados; además, el volumen se restringe al rango de 0 a 100.

Idea clave: las funciones que representan distintas partes de la aplicación llaman a Configuracion::obtenerInstancia() y trabajan con el mismo objeto.

5. Elementos del patrón utilizados

- Instancia única: variable estática local dentro de obtenerInstancia().
- Acceso global: método público estático obtenerInstancia().
- Creación controlada: constructor privado.
- Sin duplicados: constructor y operadores de copia/movimiento eliminados con = delete.
- Encapsulamiento: preferencias privadas y métodos públicos para consultarlas o modificarlas.

6. Desarrollo e implementación

La implementación se encuentra en main.cpp. La inicialización local estática de C++11 se realiza una sola vez y es segura ante llamadas concurrentes. El método de configuración modifica los valores y otra función los muestra, simulando dos módulos distintos que comparten preferencias.

Fragmento principal: acceso a la única instancia

```C/C++
static Configuracion& obtenerInstancia() {
    static Configuracion instancia;
    return instancia;
}

private:
    Configuracion() = default;
```

El constructor privado evita la creación directa de objetos. La instancia estática se conserva durante la ejecución del programa y cada llamada retorna una referencia a ella.

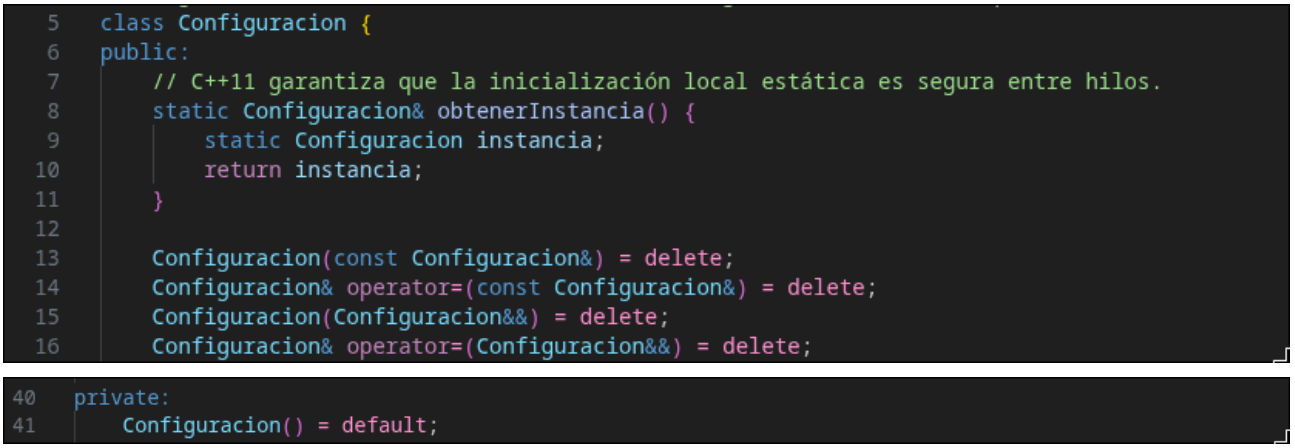


Figura 1. Implementación del punto de acceso Singleton y control de creación.

Encapsulamiento de las preferencias

```C/C++
private:
    std::string tema_ = "Claro";
    int volumen_ = 70;
    bool notificaciones_ = true;

public:
    const std::string& obtenerTema() const;
    void establecerTema(const std::string& tema);
    int obtenerVolumen() const;
    void establecerVolumen(int volumen);
```

El código completo también contiene los métodos relacionados con notificaciones y la función mostrar(). El setter del volumen corrige valores menores que 0 o mayores que 100.

```
    const std::string& obtenerTema() const { return tema_; }
    void establecerTema(const std::string& tema) { tema_ = tema; }

    int obtenerVolumen() const { return volumen_; }
    void establecerVolumen(int volumen) {
        if (volumen < 0) volumen_ = 0;
        else if (volumen > 100) volumen_ = 100;
        else volumen_ = volumen;
    }

    bool obtenerNotificaciones() const { return notificaciones_; }
    void establecerNotificaciones(bool activadas) {
        notificaciones_ = activadas;
    }
```

```
private:
    Configuracion() = default;

    std::string tema_ = "Claro";
    int volumen_ = 70;
    bool notificaciones_ = true;
};
```

Figura 2. Atributos privados y acceso controlado a la configuración.

7. Ejecución del programa

El programa muestra primero las preferencias predeterminadas. Después, configurarPreferencias() cambia los valores a tema oscuro, volumen 85 y notificaciones desactivadas. Finalmente se consulta la configuración desde otra función y se comparan dos referencias obtenidas mediante el Singleton.

Salida esperada

```
Preferencias iniciales:
Tema: Claro
Volumen: 70%
Notificaciones: activadas

Actualizando preferencias...

Preferencias desde otra parte de la aplicación:
Tema: Oscuro
Volumen: 85%
Notificaciones: desactivadas

¿Es la misma instancia? Sí
```

La última línea demuestra que ambas llamadas devuelven la misma instancia. La salida se presenta como referencia para la ejecución del programa.

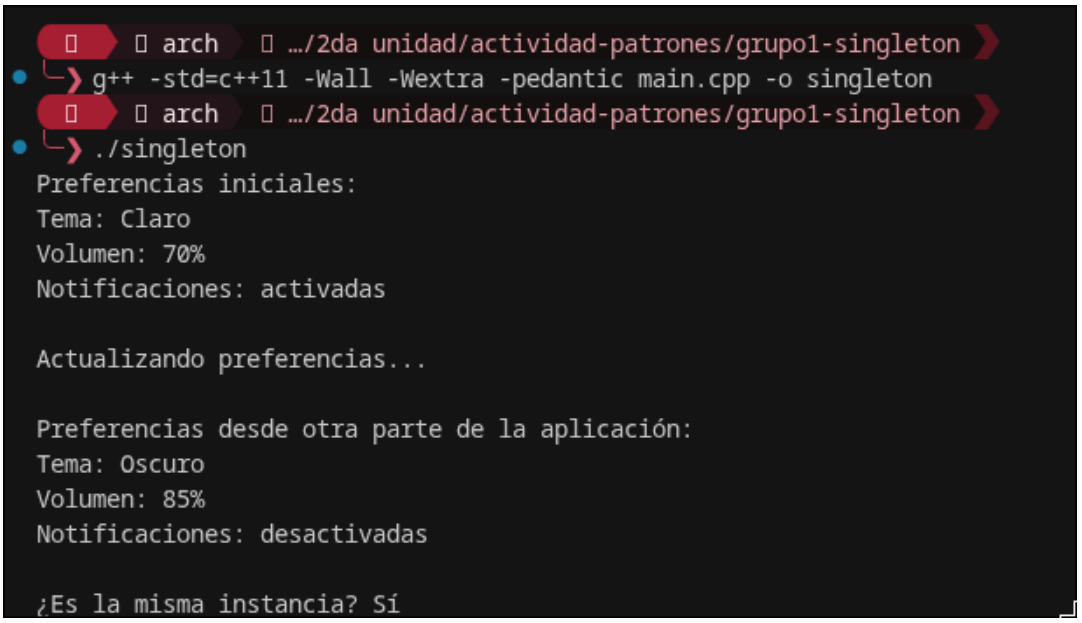


Figura 3. Ejecución del programa y comprobación de la instancia única.

8. Compilación

Desde la carpeta que contiene main.cpp, compilar con:

```
g++ -std=c++11 -Wall -Wextra -pedantic main.cpp -o singleton
./singleton
```

9. Ventajas y consideración

- Evita tener distintas copias de una configuración que debe ser compartida.
- Ofrece un punto de acceso sencillo desde distintas partes del programa.
- La inicialización local estática simplifica la creación única y es segura entre hilos desde C++11.
- El acceso global puede aumentar el acoplamiento entre componentes; conviene reservar Singleton para casos donde realmente se necesita una instancia compartida.

10. Conclusiones

- Se implementó Singleton mediante una clase con constructor privado y un método estático de acceso.
- Las funciones del ejemplo consultan y actualizan las mismas preferencias, mostrando el propósito de una instancia única.
- Los atributos privados, los métodos de acceso y la validación del volumen aplican encapsulamiento.
- La comparación de referencias permite observar que las llamadas al método de acceso retornan el mismo objeto.

11. Anexo A: Código fuente completo

```
C/C++
#include <iostream>
#include <string>
```

```cpp
// Singleton: existe una sola instancia de Configuracion en toda la aplicación.
class Configuracion {
public:
    // C++11 garantiza que la inicialización local estática es segura entre hilos.
    static Configuracion& obtenerInstancia() {
        static Configuracion instancia;
        return instancia;
    }

    Configuracion(const Configuracion&) = delete;
    Configuracion& operator=(const Configuracion&) = delete;
    Configuracion(Configuracion&&) = delete;
    Configuracion& operator=(Configuracion&&) = delete;

    const std::string& obtenerTema() const { return tema_; }
    void establecerTema(const std::string& tema) { tema_ = tema; }

    int obtenerVolumen() const { return volumen_; }
    void establecerVolumen(int volumen) {
        if (volumen < 0) volumen_ = 0;
        else if (volumen > 100) volumen_ = 100;
        else volumen_ = volumen;
    }

    bool obtenerNotificaciones() const { return notificaciones_; }
    void establecerNotificaciones(bool activadas) {
        notificaciones_ = activadas;
    }

    void mostrar() const {
        std::cout << "Tema: " << tema_ << '\n'
                  << "Volumen: " << volumen_ << "%\n"
                  << "Notificaciones: "
                  << (notificaciones_ ? "activadas" : "desactivadas") << "\n";
    }

private:
    Configuracion() = default;

    std::string tema_ = "Claro";
    int volumen_ = 70;
    bool notificaciones_ = true;
};

void configurarPreferencias() {
    Configuracion& config = Configuracion::obtenerInstancia();
    config.establecerTema("Oscuro");
    config.establecerVolumen(85);
    config.establecerNotificaciones(false);
}
```

```
void mostrarPreferencias() {
   const Configuracion& config = Configuracion::obtenerInstancia();
   config.mostrar();
}

int main() {
   std::cout << "Preferencias iniciales:\n";
   mostrarPreferencias();

   std::cout << "\nActualizando preferencias...\n";
   configurarPreferencias();

   std::cout << "\nPreferencias desde otra parte de la aplicación:\n";
   mostrarPreferencias();

   Configuracion& primera = Configuracion::obtenerInstancia();
   Configuracion& segunda = Configuracion::obtenerInstancia();
   std::cout << "\n¿Es la misma instancia? "
             << (&primera == &segunda ? "Sí" : "No") << '\n';
   return 0;
}
```